Marzo 2026

USAC

INFORME DE DESAROLLO

Lenguajes Formales y de Programación

Práctica Única

Allan Acabal

Marzo 2026

USAC

INFORME DE DESAROLLO

Lenguajes Formales y de Programación

Práctica Única

Allan Acabal

Introducción

El presente informe documenta el desarrollo de una aplicación de consola en C++ diseñada para el análisis de datos académicos provenientes de archivos con formato .lfp. El sistema permite cargar información de estudiantes, cursos y notas, realizar cálculos estadísticos avanzados (media, mediana, desviación estándar, percentiles) y generar reportes en formato HTML con los resultados obtenidos. Este proyecto es de la asignatura de Lenguajes Formales y de Programación, con el objetivo de simular las fases iniciales de un compilador: lectura, separación, validación y análisis de datos estructurados.

La aplicación se ha desarrollado siguiendo estrictamente las restricciones de uso exclusivo de la biblioteca estándar de C++ (STL), empleando técnicas manuales de manejo de cadenas para la separación de campos y estructuras de control para el procesamiento. El resultado es una herramienta funcional que cumple con los requisitos establecidos de la práctica y ofrece una interfaz común pero funcional.

Implementación

Arquitectura del Programa:

El sistema está estructurado en torno a tres entidades fundamentales:

- Estudiante
- Curso
- Nota

Representadas mediante estructuras (struct) en C++. Los datos se almacenan en arreglos estáticos con tamaños máximos definidos (MAX_ESTUDIANTES, MAX_CURSOS, MAX_NOTAS) para simplificar la gestión de memoria y evitar el uso de memoria dinámica compleja. La lógica del programa se organiza en módulos separados:

- **Estructuras.h:** Definición de las entidades.
- **Funciones.h / Funciones.cpp:** Implementación de la carga de archivos, utilidades de búsqueda y funciones estadísticas.
- **Reportes.h / Reportes.cpp:** Implementación de los cinco reportes HTML solicitados.
- **main.cpp:** Contiene el menú principal y el flujo de control.

El programa sigue un bucle principal que muestra un menú con nueve opciones, permitiendo al usuario cargar archivos, generar reportes o salir.

Lectura y Carga de Archivos:

La lectura de los archivos .lfp se realiza mediante la clase ifstream de la STL. Para cada tipo de archivo (**estudiantes.lfp, cursos.lfp, notas.lfp**) se ha implementado una función específica (**cargarEstudiantes, cargarCursos, cargarNotas**) que:

1. Abre el archivo en modo lectura.
2. Verifica su existencia; si no existe, muestra un error y retorna.

3. Lee el archivo línea por línea usando getline.
4. Para cada línea no vacía, separa los campos utilizando stringstream y getline con el delimitador ',' (coma). Este proceso equivale a un split manual.
5. Convierte los campos a los tipos de datos adecuados (stoi para enteros, stod para decimales).
6. Almacena cada registro en el arreglo correspondiente y actualiza el contador de elementos cargados.
7. Cierra el archivo al finalizar.

Cálculos Estadísticos:

Las funciones estadísticas se implementaron en Funciones.cpp y operan sobre arreglos de valores double. Se desarrollaron los siguientes cálculos:

- **Media:** Suma de todos los valores dividida por la cantidad.
- **Mediana:** Ordenamiento del arreglo (mediante sort) y selección del valor central (o promedio de los dos centrales si la cantidad es par).
- **Desviación estándar:** Cálculo de la raíz cuadrada de la varianza (poblacional, con división entre N).
- **Percentiles:** Ordenamiento y aplicación de la fórmula de interpolación lineal para obtener el valor en la posición correspondiente.
- **Máximo y mínimo:** Mediante max_element y min_element de la STL.

Además, se creó una función auxiliar extraerValoresNotas que convierte un arreglo de Nota en un arreglo de double con las calificaciones, facilitando la aplicación de las funciones estadísticas.

Generación de Reportes (HTML):

Cada reporte tiene su propia función en Reportes.cpp. El proceso general es:

1. Verificar que los datos necesarios estén cargados (por ejemplo, para el reporte por curso se requieren cursos y notas).
2. Abrir un archivo de salida con extensión .html usando ofstream.
3. Escribir la cabecera HTML con metadatos y estilos CSS básicos para mejorar la presentación.
4. Recorrer las estructuras de datos (cursos, estudiantes, etc.) y, para cada elemento, realizar los cálculos necesarios (promedios, conteos, porcentajes).
5. Generar filas de tabla con los resultados formateados (fixed y setprecision para decimales).
6. Cerrar el archivo y notificar al usuario.

Los reportes generados son:

- **Estadísticas por Curso:** Muestra código, nombre, cantidad de estudiantes, promedio, máxima, mínima, desviación estándar y mediana.
- **Rendimiento por Estudiante:** Para cada estudiante: nombre, carnet, carrera, semestre, promedio general, aprobados, reprobados y créditos acumulados.
- **Top 10 Mejores Estudiantes:** Lista ordenada con posición, datos personales y promedio.
- **Cursos con Mayor Reprobación:** Ordenados por porcentaje de reprobados, incluyendo totales y porcentaje.
- **Análisis por Carrera:** Para cada carrera, muestra total de estudiantes, promedio general, cursos disponibles y distribución por semestre.

Manejo de Errores:

El menú principal se implementa en main.cpp mediante un bucle do-while que muestra las opciones y lee la entrada del usuario. Se ha incluido manejo básico de errores:

- **Validación de opciones:** si el usuario ingresa un número fuera del rango 1-9 ó o algún texto, se muestra un mensaje de error y se vuelve a pedir la opción.
- **Verificación de existencia de archivos:** las funciones de carga notifican si el archivo no pudo abrirse.
- **Comprobación de datos previos a la generación de reportes: se** verifica que los arreglos no estén vacíos; de lo contrario, se muestra un mensaje indicando la necesidad de cargar los archivos correspondientes.

Problemas Ocurridos y Soluciones:

1. Separación de campos sin función split nativa en C++

Diagnóstico:
C++ no cuenta con una función split() incorporada en su biblioteca estándar, a diferencia de otros lenguajes como Python o Java. Esto obliga a implementar manualmente la separación de los campos en cada línea de los archivos .lfp. Sin un método adecuado, es fácil cometer errores como no considerar comas dentro de cadenas o saltos de línea inesperados.

Solución:
Se utilizó la combinación de stringstream y getline con el delimitador ',' para extraer cada campo de forma secuencial. Esto permite recorrer la línea y almacenar cada valor en variables temporales. Además, se ignoraron líneas vacías y se recortaron espacios en blanco (aunque en los datos de ejemplo no hay espacios, es una buena práctica considerarlos).

2. Gestión de memoria con arreglos estáticos

Diagnóstico:
Luego de unos intentos decidí usar arreglos estáticos en lugar de vectores (vector), no tenía dominado el uso de ellos, me surgió una duda, la cual era si los archivos contienen más registros que los definidos en las constantes (MAX_ESTUDIANTES, MAX_CURSOS, MAX_NOTAS). Además, se debe llevar un control preciso del número de elementos realmente cargados para evitar accesos fuera de rango.

Solución:
Se definieron constantes con valores suficientemente grandes para cubrir casos típicos (100 estudiantes, 50 cursos, 500 notas). En las funciones de carga, se verifica que el contador actual no supere el límite antes de agregar un nuevo elemento; si se alcanza el máximo, se detiene la lectura y se muestra una advertencia. Todas las funciones que recorren los arreglos reciben como parámetro el tamaño real, evitando así accesos indebidos.

3. Cálculo correcto de mediana y percentiles

Diagnóstico:
La mediana y los percentiles requieren que los datos estén ordenados. Si se ordena el arreglo original, se pierde la información de orden para otros cálculos. Además, la fórmula de percentiles no es trivial, especialmente cuando la posición teórica no es un número entero, y se necesita interpolación lineal.

Solución:
Se implementaron las funciones mediana y percentil trabajando sobre copias de los arreglos de valores. De esta forma, los datos originales permanecen intactos. Para la mediana, se ordena la copia y se selecciona el valor central (o el promedio de los dos centrales). Para los percentiles, se aplica la fórmula estándar: **pos = p * (n-1) / 100**, luego se interpola entre los dos valores más cercanos.

4. Relación entre datos de múltiples archivos

Diagnóstico:
Uno de los mayores desafíos lógicos fue cruzar la información de los tres archivos: estudiantes, cursos y notas. Por ejemplo, para obtener el promedio de un estudiante, es necesario recolectar todas sus notas, y luego asociar cada nota con el curso correspondiente para calcular créditos acumulados.

Solución:
Se crearon funciones de utilidad específicas:

- **notasDeEstudiante:** filtra el arreglo de notas por carnet y devuelve un arreglo con las notas de ese estudiante.
- **notasDeCurso**: similar, pero filtra por código de curso.
- **buscarIndiceEstudiante** y **buscarIndiceCurso**: devuelven la posición de un estudiante o curso en sus respectivos arreglos, permitiendo acceder rápidamente a sus atributos.

Resultados Finales:

Menú Principal:

Acá se muestra la estructura general y final que se aplicó para el menú de la práctica:

```
==========================================
 SISTEMA DE ANALISIS ACADEMICO
==========================================
1. Cargar archivo de estudiantes
2. Cargar archivo de cursos
3. Cargar archivo de notas
4. Generar Reporte: Estadisticas por Curso
5. Generar Reporte: Rendimiento por Estudiante
6. Generar Reporte: Top 10 Mejores Estudiantes
7. Generar Reporte: Cursos con Mayor Reprobacion
8. Generar Reporte: Analisis por Carrera
9. Salir
Seleccione una opcion:
```

Carga de Archivos:

El menú despliega 3 opciones de carga de archivos:

- Opción1: Estudiantes
- Opción 2: Notas
- Opción 3: Cursos

Al elegir çada opción despliega mensajes como estos:

```
Seleccione una opcion: 1
Estudiantes cargados: 16
```

```
Seleccione una opcion: 2
Cursos cargados: 8
```

```
Seleccione una opcion: 3
Notas cargadas: 16
```

Todos los datos de los archivos cargados, se almacenan en archivos.flp, con su respectivo nombre.

Reportes HTML:

Reporte 1: Estadísticas Generales por Curso

Estadísticas Generales por Curso

Código	Nombre del Curso	Estudiantes	Promedio	Máxima	Mínima	Desv. Est.	Mediana
771	Estructuras de Datos	2	73.00	81.00	65.00	8.00	73.00
773	Sistemas Operativos 1	2	71.50	84.00	59.00	12.50	71.50
774	Bases de Datos 1	2	80.25	92.00	68.50	11.75	80.25
349	Mecánica de Fluidos	2	80.00	90.00	70.00	10.00	80.00
350	Análisis Estructural 1	2	75.75	91.50	60.00	15.75	75.75
102	Cálculo II	2	71.50	88.00	55.00	16.50	71.50
103	Física I	2	61.50	74.00	49.00	12.50	61.50
104	Estadística 1	2	64.50	79.00	50.00	14.50	64.50

Reporte 2: Rendimiento por Estudiante

Rendimiento por Estudiante

Carnet	Nombre Completo	Carrera	Semestre	Promedio General	Aprobados	Reprobados	Créditos Acum.
202016789	Pedro Ramírez	Sistemas	6	81.00	1	0	5
202017890	Luisa Hernández	Industrial	4	74.00	1	0	4
202018901	Diego Castillo	Civil	8	90.00	1	0	6
202019012	Sofía Morales	Sistemas	3	68.50	1	0	5
202020123	Andrés Reyes	Industrial	5	88.00	1	0	4
202021234	Valeria Ortiz	Civil	6	91.50	1	0	6
202022345	José Chávez	Sistemas	7	84.00	1	0	5
202023456	Camila Navarro	Industrial	2	50.00	0	1	0
202024001	María Gómez	Sistemas	2	65.00	1	0	5
202024002	Carlos López	Industrial	3	55.00	0	1	0
202024003	Ana Martínez	Civil	4	70.00	1	0	6
202024004	Jorge Pérez	Sistemas	1	92.00	1	0	5
202024005	Daniela Ruiz	Industrial	6	79.00	1	0	4
202024006	Miguel Torres	Civil	5	60.00	0	1	0
202024007	Paula Jiménez	Sistemas	4	59.00	0	1	0
202024008	Roberto Sánchez	Industrial	7	49.00	0	1	0

Reporte 3: Top 10 Mejores Estudiantes

Top 10 Mejores Estudiantes

Posición	Carnet	Nombre Completo	Carrera	Semestre	Promedio
1	202024004	Jorge Pérez	Sistemas	1	92.00
2	202021234	Valeria Ortiz	Civil	6	91.50
3	202018901	Diego Castillo	Civil	8	90.00
4	202020123	Andrés Reyes	Industrial	5	88.00
5	202022345	José Chávez	Sistemas	7	84.00
6	202016789	Pedro Ramírez	Sistemas	6	81.00
7	202024005	Daniela Ruiz	Industrial	6	79.00
8	202017890	Luisa Hernández	Industrial	4	74.00
9	202024003	Ana Martínez	Civil	4	70.00
10	202019012	Sofía Morales	Sistemas	3	68.50

Reporte 4: Cursos con Mayor Índice de Reprobación

Cursos con Mayor Índice de Reprobación

Código	Nombre del Curso	Total Estudiantes	Aprobados	Reprobados	% Reprobación
773	Sistemas Operativos 1	2	1	1	50.00%
350	Análisis Estructural 1	2	1	1	50.00%
102	Cálculo II	2	1	1	50.00%
103	Física I	2	1	1	50.00%
104	Estadística 1	2	1	1	50.00%
774	Bases de Datos 1	2	2	0	0.00%
349	Mecánica de Fluidos	2	2	0	0.00%
771	Estructuras de Datos	2	2	0	0.00%

Reporte 5: Análisis por Carrera

Análisis por Carrera

Carrera: Sistemas

Total Estudiantes	6
Promedio General	74.92
Cursos Disponibles	3

Distribución por Semestre

Semestre	Cantidad Estudiantes
1	1
2	1
3	1
4	1
6	1
7	1

Carrera: Industrial

Total Estudiantes	6
Promedio General	65.83
Cursos Disponibles	3

Distribución por Semestre

Semestre	Cantidad Estudiantes
2	1
3	1
4	1
5	1
6	1
7	1

Carrera: Civil

Total Estudiantes	4
Promedio General	77.88
Cursos Disponibles	2

Distribución por Semestre

Semestre	Cantidad Estudiantes
4	1
5	1
6	1
8	1

Manejo de Entradas Inválidas

1. Restricciones en letras:

Al ingresar un valor de texto, el programa muestra un mensaje de error y permite volver a ingresar un valor (Este debe ser número).

```
Seleccione una opcion: d
Entrada invalida. Por favor ingrese un numero: 1
```

2. Restricciones en números fuera de rango:

Al ingresar un número fuera del rango (1-9), este le indica al usuario que fue una opción incorrecta y le permite interactuar de nuevo con el menú.

```
Seleccione una opcion: 10
Opcion fuera de rango. Intente de nuevo.
```

Conclusiones

El desarrollo de este sistema de análisis académico ha permitido aplicar conceptos fundamentales de programación en C++, como manejo de archivos, estructuras de datos, algoritmos de ordenamiento y cálculos estadísticos. Se ha cumplido con todos los requisitos funcionales: lectura de archivos en formato .lfp, separación manual de campos, almacenamiento en arreglos, generación de reportes HTML y un menú interactivo.

Uno de los principales logros fue la implementación de funciones estadísticas sin recurrir a librerías externas, lo que demuestra un dominio de los fundamentos matemáticos y de programación. La relación entre los tres archivos (estudiantes, cursos, notas) se resolvió mediante funciones de búsqueda y filtrado, permitiendo obtener información cruzada de manera eficiente.

Como resultado, se obtuvo una aplicación funcional y bien documentada, que sirve como base para futuras ampliaciones, como la inclusión de gráficos en los reportes o la mejora del manejo de errores.